

Chapter 2

Object Detection

In standard image classification tasks, we operate under the assumption that there is only one major object in the image, allowing us to focus entirely on recognizing its category. However, the real world is rarely that simple. Images typically contain multiple objects of interest interacting within a complex scene. To build robust computer vision systems, we must not only determine the categories of these objects but also pinpoint their specific positions within the image. This dual-faceted challenge of localization and classification is known as object detection.

This chapter transitions from basic image classification to the intricate mechanics of object detection. We will begin by exploring the foundational representations of spatial locations—such as bounding boxes—and discuss naive approaches to finding objects. From there, we will build a critical toolkit of evaluation metrics and algorithms, including Intersection over Union (IoU), Non-Maximum Suppression (NMS), and Mean Average Precision (mAP). Finally, we will trace the architectural evolution of object detection models, contrasting the highly accurate, two-stage Region-based Convolutional Neural Networks (the R-CNN family) with the blazingly fast, single-stage detectors like YOLO. By the end of this chapter, you will have a comprehensive understanding of how modern AI systems perceive, localize, and classify the world around them.

2.1 Introduction

2.1.1 Definition of the Problem

In object detection, we not only want to know the categories of the objects, but also their specific **positions** in the image, as shown in Figure 2.1. Object detection has been widely applied in many fields. For example, self-driving needs to plan traveling routes by detecting the positions of vehicles, pedestrians, and obstacles in the captured video images. Besides, robots may use this technique to detect and localize objects of interest throughout its navigation of an environment.

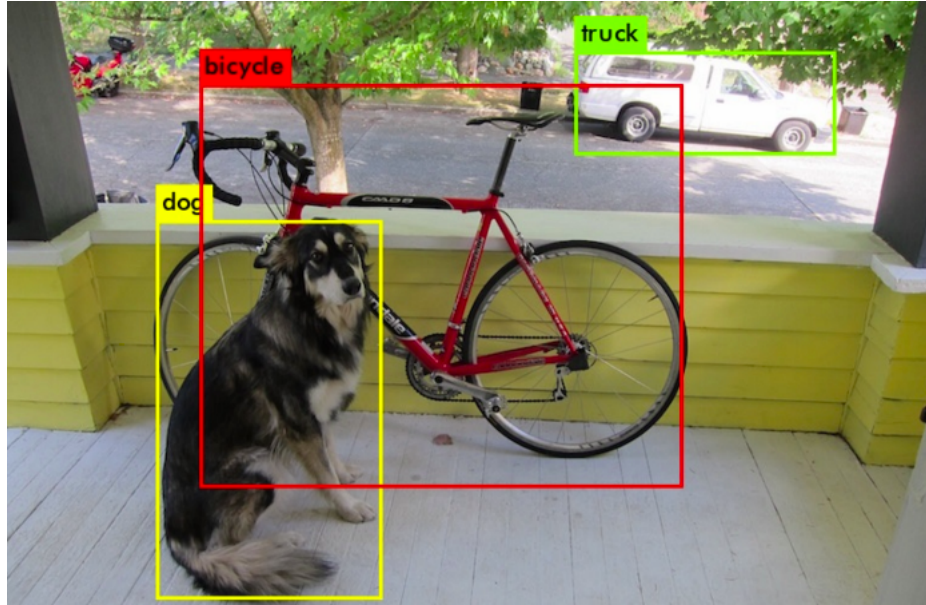


Figure 2.1: Example of object detection.

In object detection, we usually use a **bounding box** to describe the spatial location of an object. The bounding box is rectangular, which is determined by the x and y coordinates of the upper-left corner of the rectangle and the such coordinates of the lower-right corner. Another commonly used bounding box representation is the (x, y) -axis coordinates of the bounding box center, and the width and height of the box. An example of both is shown in Figure 2.2.

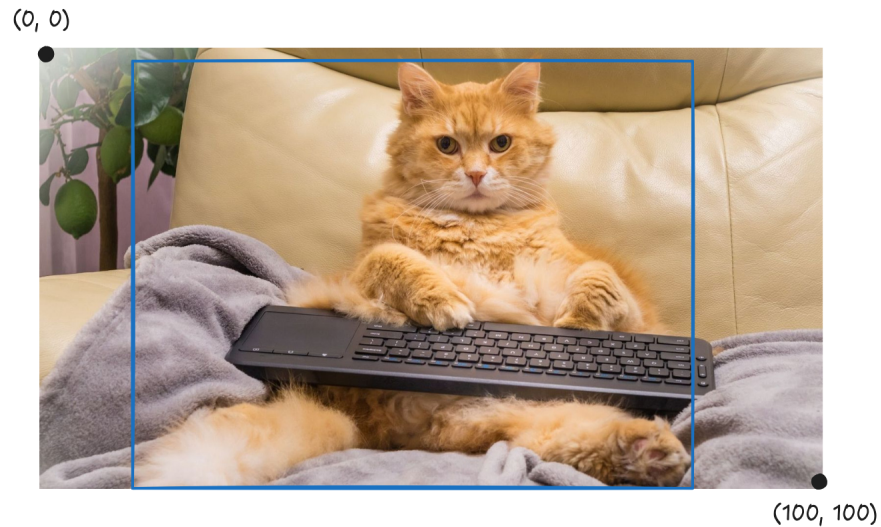


Figure 2.2: The bounding box of the cat could be represented with the x and y coordinates of the upper-left corner of the rectangle and the such coordinates of the lower-right corner like $(x_1, y_1, x_2, y_2) = (10, 5, 85, 100)$ or with the (x, y) -axis coordinates of the bounding box center, and the width and height of the box like $(x, y, w, h) = (47.5, 52.5, 75, 95)$.

2.1.2 Detecting a Single Object

If we want to detect a single object, there is a simple method. We can pass the image through a CNN and then use two MLPs, like in Figure 2.3: the first one to obtain the class of the object and second one to obtain the coordinates of the box. Finally, the loss would be the weighted sum of the Cross Entropy (or Binary Cross Entropy if there is only one object¹) Loss of the classification and the L_1 or L_2 Loss of the predicted coordinates. Note that one class must be reserved for “**no object**” or **background** class, and in this case the coordinates of the bounding box would be irrelevant.

Formally, if there are n classes, the true label is the one-hot vector $y = (y_1, \dots, y_n)^T$ where $y_i = 1$ if the i -th class is the true class and 0 otherwise, the logits predicted are $\hat{y} = (\hat{y}_1, \dots, \hat{y}_n)^T$, the true coordinates of the bounding box are (x_1, y_1, x_2, y_2) and the predicted coordinates are $(\hat{x}_1, \hat{y}_1, \hat{x}_2, \hat{y}_2)$, the loss is:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \cdot \mathbb{1}_{\text{no_bg}} \cdot \mathcal{L}_{L_2},$$

where

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^n \log \left(\frac{\exp(\hat{y}_i)}{\sum_{j=1}^n \exp(\hat{y}_j)} \right) y_i,$$

$$\mathcal{L}_{L_2} = \frac{1}{4} \left((x_1 - \hat{x}_1)^2 + (y_1 - \hat{y}_1)^2 + (x_2 - \hat{x}_2)^2 + (y_2 - \hat{y}_2)^2 \right),$$

$\mathbb{1}_{\text{no_bg}}$ is a binary indicator that takes the value of 1 if the true class is different from the background and 0 otherwise (this way we only take into account the coordinates loss if an object is present), and $\lambda > 0$ a constant to control the weight for the coordinates loss.

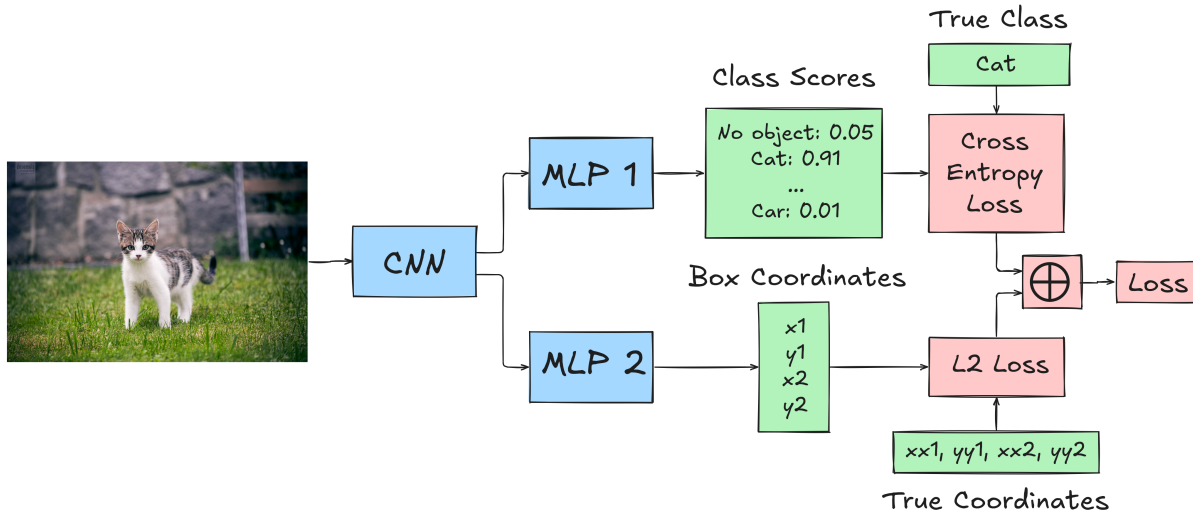


Figure 2.3: Detecting a single object.

¹A 0 would represent that there is no object detected, and a 1 that an object is detected. Note that this way we can only identify objects of one single class.

Note

If we only want to detect one single object of the same class and we know it will appear in all the images, we can use a CNN and just one MLP for the coordinates of the bounding box and it would only be a regression problem. This could be useful, for example, if we want to track the position of the ball in a football match. We are not interested in players, goalkeepers or the referee, only in the ball. We would use the model for each frame and obtain the position of the ball throughout the match.

2.1.3 Naive Approach

The problem of the previous approach is that we are constrained to predicting just one object, but images usually have more than one. A **naive approach** is to create different crops of the image (like in Figure 2.4), and use an object detection model, for example the one already explained, to classify each crop.

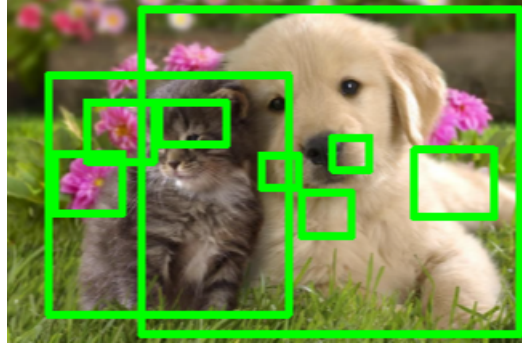


Figure 2.4: Many crops for an image.

However, there is a big problem: the number of possible boxes increases very fast with the width and height of the image. Consider an image of size $H \times W$ and let the first index be 1 instead of 0 for this example. A bounding box is determined by two rows r_1, r_2 and two columns c_1, c_2 with $1 \leq r_1 \leq r_2 \leq H$ and $1 \leq c_1 \leq c_2 \leq w$. Now:

- To select the rows:
 - For $r_1 = 1$ there are W options for r_2 ($1, 2, \dots, W$).
 - For $r_1 = 2$ there are $W - 1$ options for r_2 ($2, 3, \dots, W$).
 - ...
 - For $r_1 = W$ there is 1 option for R_2 (W).

Therefore, there is a total of $\sum_{i=1}^W i = \frac{W(W+1)}{2}$ possible selections.

- To select the columns:
 - For $c_1 = 1$ there are H options for c_2 ($1, 2, \dots, H$).

- For $c_1 = 2$ there are $H - 1$ options for c_2 ($2, 3, \dots, H$).
- ...
- For $c_1 = H$ there is 1 option for c_2 (H).

Therefore, there is a total of $\sum_{i=1}^H i = \frac{H(H+1)}{2}$ possible selections.

- In conclusion, there is a total of $\frac{H(H+1)}{2} \cdot \frac{W(W+1)}{2}$ possible selections.

Try to calculate by hand all possible bounding boxes for a 3×3 image and check that the answer is $(3 \cdot 4/2) \cdot (3 \cdot 4/2) = 36$.

There is no way we can evaluate all this boxes. Even if we select randomly a fixed number of boxes per pixel, for example c , the total number of boxes would be roughly² cHW , still a lot, and we take the risk of not selecting the correct ones.

2.2 Previous Concepts

This section introduces some key concepts in order to solve the problem correctly. Intersection over Union, Non-Max Suppression and Mean Average Precision are metrics and topics we need to understand before constructing the final solution. Also, Selective Search is an algorithm used to propose regions that are likely to contain objects of interest.

2.2.1 Intersection over Union (IoU)

If the ground-truth bounding box of the object is known, how can a “good” predicted bounding box here be **quantified**? Intuitively, we can measure the similarity between the predicted box and the ground-truth bounding box. We know that the Jaccard index can measure the similarity between two sets. Given sets A and B , their Jaccard index is the size of their intersection divided by the size of their union:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

In fact, we can consider the pixel area of any bounding box as a set of pixels. In this way, we can measure the similarity of the two bounding boxes by the Jaccard index of their pixel sets. For two bounding boxes, we usually refer their Jaccard index as **Intersection over Union (IoU)**, which is the ratio of their intersection area to their union area. The range of IoU is between 0 and 1: 0 means that two bounding boxes do not overlap at all, while 1 indicates that the two bounding boxes are equal. An example is shown in Figure 2.5, where the IoU is computed for many different bounding boxes.

²Roughly because in some pixels $(x, y) \in [1, \dots, H] \times [1, \dots, W]$ there may be less than c possible bounding boxes.



Figure 2.5: IoU calculated for different bounding boxes.

2.2.2 Non-Maximum Suppression (NMS)

When there are many similar (with significant overlap) predicted bounding boxes, they can be potentially output for surrounding the same object. To simplify the output of the network, we can merge similar predicted bounding boxes that belong to the same object by using **Non-Maximum Suppression (NMS)**. An example is shown in Figure 2.6.

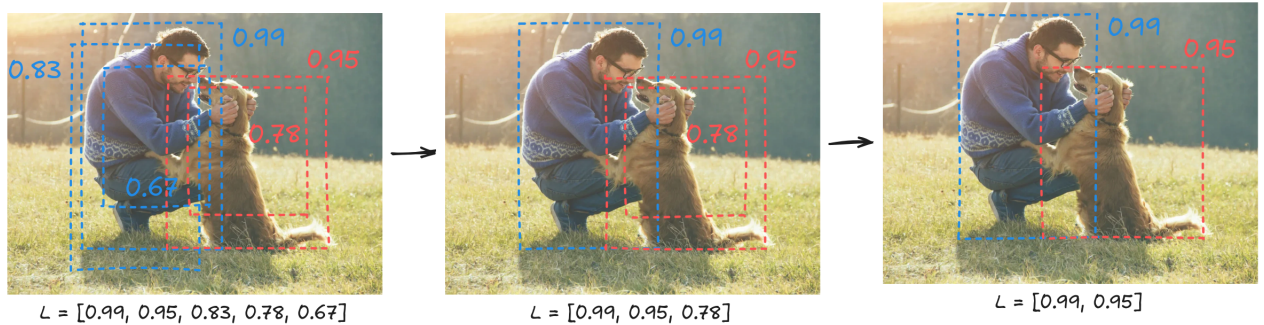


Figure 2.6: NMS process for an image with two classes: person (blue) and dog (red).

For a predicted bounding box B , the object detection model calculates the predicted likelihood for each class. Denoting by p the largest predicted likelihood, the class corresponding to this probability is the predicted class for B . Specifically, we refer to p as the confidence (score) of the predicted bounding box B . On the same image, all the predicted non-background bounding boxes are sorted by confidence in descending order to generate a list L . Then we manipulate the sorted list L in the following steps:

1. Select the predicted bounding box with the highest confidence from L that has not been previously selected and remove all predicted bounding boxes whose IoU with the selected box exceed a predefined threshold ε from L . At this point, L keeps the selected bounding box but drops others that are too similar to it. In a nutshell, those with non-maximum confidence scores are suppressed.

2. Repeat the above process until all the predicted bounding boxes in L have been selected. At this time, the IoU of any pair of predicted bounding boxes in L is below the threshold ϵ ; thus, no pair is too similar with each other.
3. Output all the predicted bounding boxes in the list L .

Note

In step 1, you can consider that, in addition to exceeding the IoU threshold, the boxes must be labeled with the same class. In step 3, you can delete bounding boxes whose probability is below certain threshold, for example 0.5.

2.2.3 Mean Average Precision (mAP)

The **mean Average Precision (mAP)** is a widely used performance metric in information retrieval and object detection tasks in machine learning. It provides a single number that summarizes the precision-recall curve, reflecting how well a model is performing across different threshold levels. This metric is particularly useful in scenarios like object detection, where models not only need to detect the presence of objects but also accurately localize and classify them.

The **precision** is the ratio of true positive predictions out of all positive predictions made. It measures the accuracy of the positive predictions:

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

The **recall** is the ratio of true positive predictions out of all actual positive observations:

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

For one class and one fixed IoU threshold τ , use this procedure:

1. Gather that class's detections across the evaluation set and sort them by descending confidence.
2. For each detection in that order, consider only ground-truth boxes of the same class in the same image. Match it to the highest-IoU ground-truth box that has not already been matched.
3. Mark the detection as a true positive only if that IoU is at least τ ; otherwise mark it as a false positive. A duplicate detection of an already matched object is a false positive.
4. From the cumulative TP and FP counts, calculate precision and recall after every ranked detection.

5. Apply the interpolation/integration rule required by the chosen benchmark to obtain AP (area under the precision-recall curve).
6. Average AP over classes to obtain class-macro mAP. If the protocol uses several IoU thresholds, also average over those thresholds.

Note

The integration rule must be named because benchmark definitions differ. For example, VOC 2007 used 11-point interpolated AP, later VOC evaluation used all recall changes, and COCO evaluates interpolated precision at 101 recall thresholds and averages over IoU thresholds from 0.50 to 0.95 with a step of 0.05, among other settings.

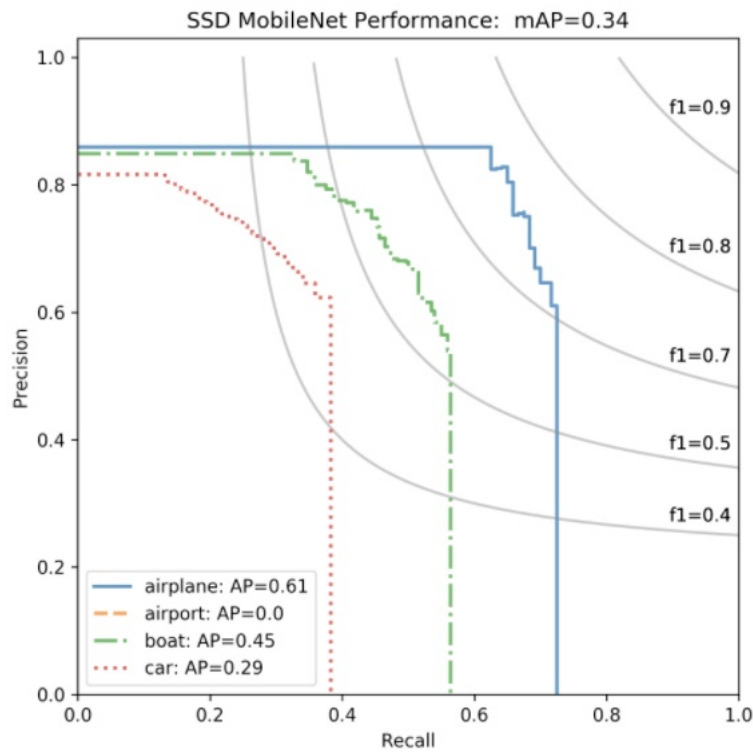


Figure 2.7: Example of precision-recall curves for different classes.

2.2.4 Selective Search

To address the problem of the naive approach we can use the **selective search**, which is an algorithm that creates a small number of region proposals in comparison. Moreover, these regions have a semantic meaning, i.e., they are not just random. We will explain a summary, but you can see the [original paper](#) for more details. The algorithm (shown in Figures 2.8, 2.9 and 2.10) is:

1. Generate initial sub-segmentation of input image using the method described by Felzenszwalb et al in his paper "[Efficient Graph-Based Image Segmentation](#)".

2. Recursively combine the smaller similar regions into larger ones. A greedy algorithm can be used to combine similar regions to make larger regions. The algorithm is:
 - (a) From set of regions, choose two that are most similar.
 - (b) Combine them into a single, larger region.
 - (c) Repeat the above steps for multiple iterations.
3. Use the segmented region proposals to generate candidate object locations.



Figure 2.8: First step of the selective search.

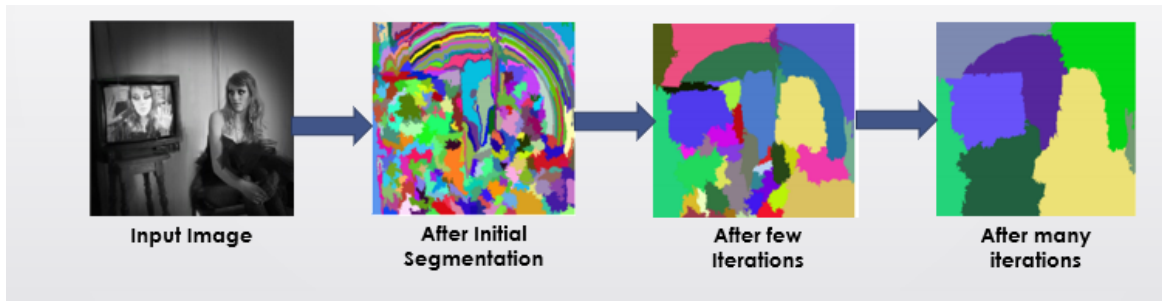


Figure 2.9: Second step of the selective search.

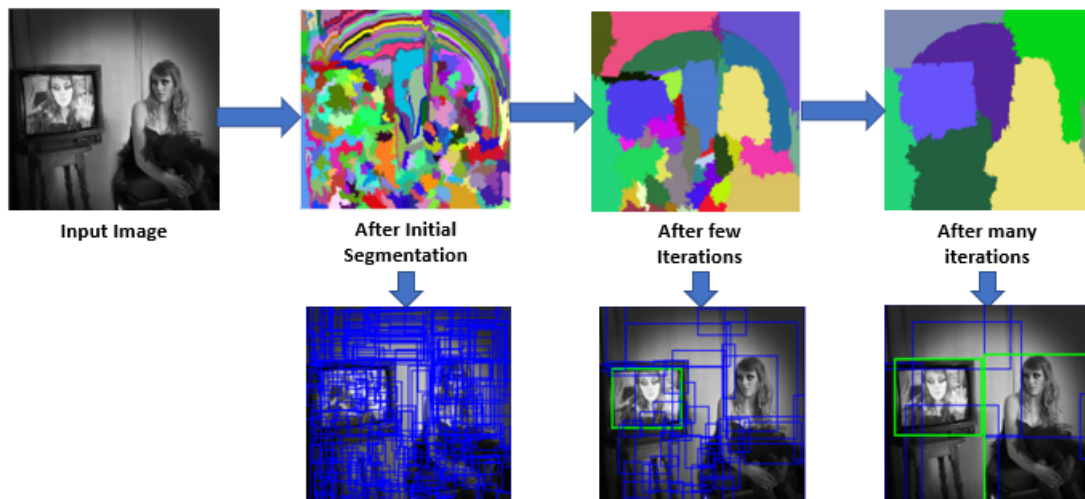


Figure 2.10: Last step of the selective search.

2.2.5 Anchor Boxes

There are more ways of generating a small number of meaningful regions. Suppose that the input image has a height of h and width of w . We generate **anchor boxes** with different shapes centered on each pixel of the image. Let the scale be $s \in (0, 1]$ and the aspect ratio (ratio of width to height) is $r > 0$. Then the width and height of the anchor box are $ws\sqrt{r}$ and $hs/\sqrt{r}$, respectively. Note that when the center position is given, an anchor box with known width and height is determined.

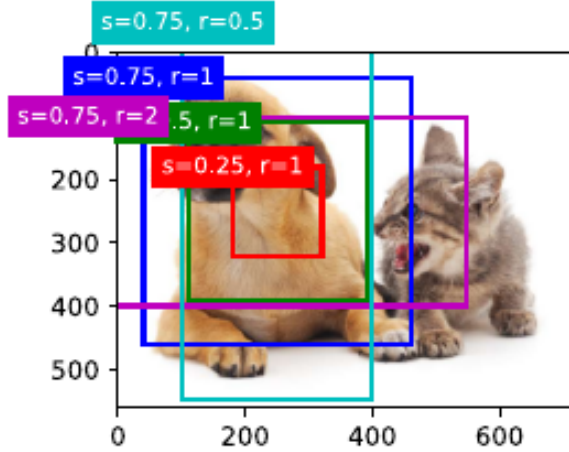


Figure 2.11: Some anchor boxes centered in a pixel of the dog.

To generate multiple anchor boxes with different shapes, let's set a series of scales $s_1, \dots, s_n$ and a series of aspect ratios $r_1, \dots, r_m$. When using all the combinations of these scales and aspect ratios with each pixel as the center, the input image will have a total of $whnm$ anchor boxes. Although these anchor boxes may cover all the ground-truth bounding boxes, the computational complexity is easily too high. In practice, we can only consider those combinations containing s_1 or r_1 :

$$(s_1, r_1), (s_1, r_2), \dots, (s_1, r_m); (s_2, r_1), (s_3, r_1), \dots, (s_n, r_1).$$

That is to say, the number of anchor boxes centered on the same pixel is $n + m - 1$. For the entire input image, we will generate a total of $wh(n + m - 1)$ anchor boxes. Typically s_1 is an intermediate scale or scale of reference and r_1 a square ratio (1:1 in case $h = w$) or a neutral ratio.

2.3 R-CNNs

The **Region-based Convolutional Neural Networks (R-CNNs)** were one of the first networks used for object detection. The algorithm is the following and an schema is shown in Figure 2.12:

1. Perform selective search to extract ~ 2000 high-quality region proposals on the input image.

2. Choose a pretrained CNN and truncate it before the output layer (you can optionally fine-tune the CNN for foreground classes plus background). Resize each region proposal to the input size required by the network, and output the extracted features for the region proposal through forward propagation.
3. Take the extracted features and labeled class of each region proposal as an example. Train multiple support vector machines (SVMs) to classify objects, where each SVM individually determines whether the example contains a specific class³.
4. Take the extracted features and labeled bounding box of each region proposal as an example. Train a linear regression model to predict the ground-truth bounding box⁴.
5. Translate the predicted bounding box coordinates to the original image.
6. Select a confidence threshold (for example, only consider region proposals with predicted class with probability greater or equal than 0.5) and use NMS to output the final predictions.

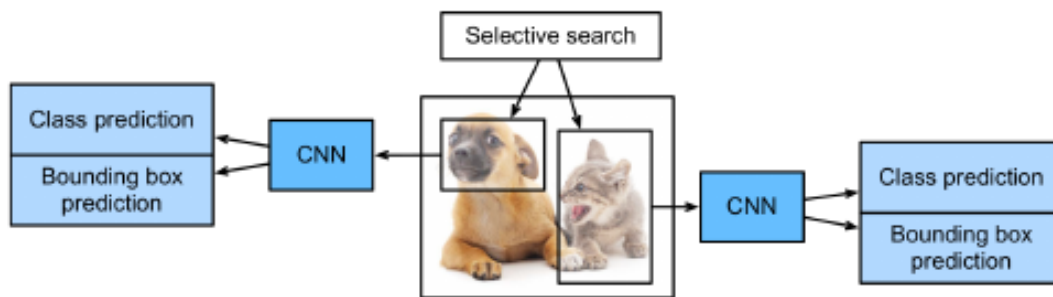


Figure 2.12: R-CNN algorithm.

Although the R-CNN model uses pretrained CNNs to effectively extract image features, it is slow. Imagine that we select thousands of region proposals from a single input image: this requires $\lceil n/b \rceil$ CNN forward propagations to perform object detection, where n is the total number of regions and b the batch size. This massive computing load makes it infeasible to widely use R-CNNs in real-world applications.

2.4 Fast R-CNNs

The main performance bottleneck of an R-CNN lies in the number CNN forward propagations without sharing computation. Since these regions usually have overlaps, independent feature

³In fact, there are many ways to deal with this step. We just want to predict the class of the object in the region proposal. For example, we could use a MLP for multi-class classification. However, the original approach was the one described using SVMs.

⁴Again, we can meet this goal in many ways, for example using a Random Forest Regressor instead of a linear regression. We can even use a model to predict at the same time the class and the coordinates of the bounding box at the same time. However, the original approach was the one described using linear regression.

extractions lead to much repeated computation. One of the major improvements of the **Fast Region-based Convolutional Neural Networks (Fast R-CNNs)** from the R-CNN is that the CNN forward propagation is only performed once on the entire image.

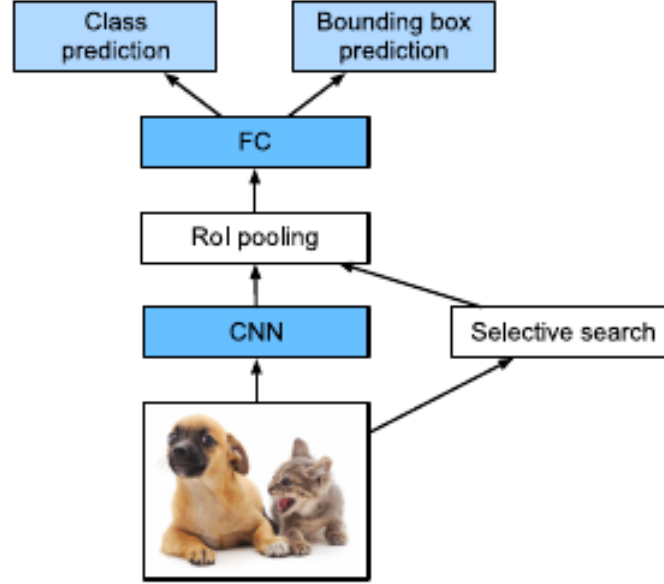


Figure 2.13: Fast R-CNN algorithm.

The algorithm is the following and an schema is shown in Figure 2.13:

1. The input of the CNN for feature extraction is the entire image, rather than individual region proposals. Moreover, this CNN is trainable. Given an input image, let the shape of the CNN output be $1 \times c \times h_1 \times w_1$.
2. Suppose that selective search generates n region proposals. These region proposals (of different shapes) mark regions of interest (of different shapes) on the CNN output.
3. These regions of interest have to extract features of the same shape (say height h_2 and width w_2 are specified) in order to be easily concatenated. To achieve this, we take the portion of the CNN feature map that corresponds to each region proposal (it will always have c channels and its height and width depends on the region proposal) and we pass it through the Region of Interest (RoI) pooling layer, outputting features of shape $c \times h_2 \times w_2$. We concatenate all this outputs and we finally have a $n \times c \times h_2 \times w_2$ tensor.
4. Using a fully connected layer, transform the concatenated features into an output of shape $n \times d$ (note that $d = c \cdot h_2 \cdot w_2$) and predict the class and bounding box for each of the n region proposals.
5. Translate the predicted bounding box coordinates to the original image.

6. Select a confidence threshold (for example, only consider region proposals with predicted class with probability greater or equal than 0.5) and use NMS to output the final predictions.

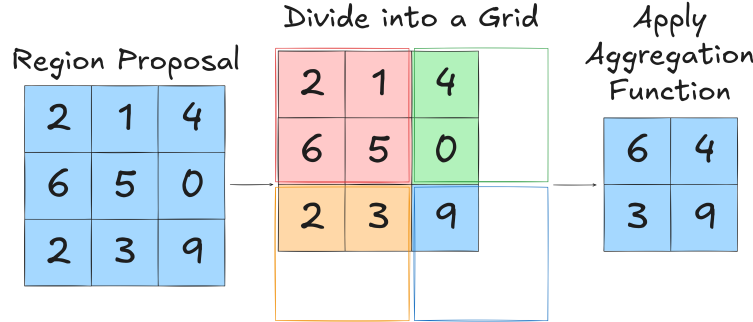


Figure 2.14: RoI pooling using the maximum as aggregation function.

The RoI pooling is a layer that transforms all region proposal into the same shape. If we want a shape $h \times w$ and the region proposal has a shape of $H \times W$, then we will divide the region proposal into a grid of cells with shape $\lceil H/h \rceil \times \lceil W/w \rceil$. Finally, an aggregation function can be used for the output, like max or mean. For example, if the region proposal has a shape of 3×3 and we want a shape of 2×2 , then we will divide the region proposal into a grid of cells $\lceil 3/2 \rceil \times \lceil 3/2 \rceil = 2 \times 2$. Note that in general most cells will be 2×2 , but near the edges the size can be smaller. This example is represented in Figure 2.14.

2.5 Faster R-CNNs

To be more accurate in object detection, the fast R-CNN model usually has to generate a lot of region proposals in selective search. To reduce region proposals runtime bottleneck, the **Faster Region-based Convolutional Neural Networks (Faster R-CNNs)** proposes to replace selective search with a **region proposal network** (steps 2, 3, 4 and 5 of the algorithm). The idea here is to try to “learn” the region proposals we need instead of just using the selective search.

The algorithm is the following and an schema is shown in Figure 2.15:

1. Use a CNN to extract the feature map of the input.
2. Apply another convolution to the feature map.
3. Centered on each pixel of the feature maps, generate multiple anchor boxes of different scales and aspect ratios and label them.
4. Using the feature map of the convolution corresponding to each anchor box, predict the binary class (background or objects) and bounding box for each anchor box.

5. Consider those predicted bounding boxes whose predicted classes are objects. Remove overlapped results using NMS. The remaining predicted bounding boxes for objects are the region proposals required by the region of interest pooling layer.
6. We take the portion of the CNN feature map that corresponds to each bounding box, pass it through the RoI pooling layer so that they are all the same size and they are concatenated. Let's suppose there are n bounding boxes left, that the CNN feature map has c channels and the desired height and width of the transformed features are h_2 and w_2 , respectively. We finally have a $n \times c \times h_2 \times w_2$ tensor.
7. Using a fully connected layer, transform the concatenated features into an output of shape $n \times d$ (note that $d = c \cdot h_2 \cdot w_2$) and predict the class and bounding box for each of the n region proposals.
8. Translate the predicted bounding box coordinates to the original image.
9. Select a confidence threshold (for example, only consider region proposals with predicted class with probability greater or equal than 0.5) and use NMS to output the final predictions.

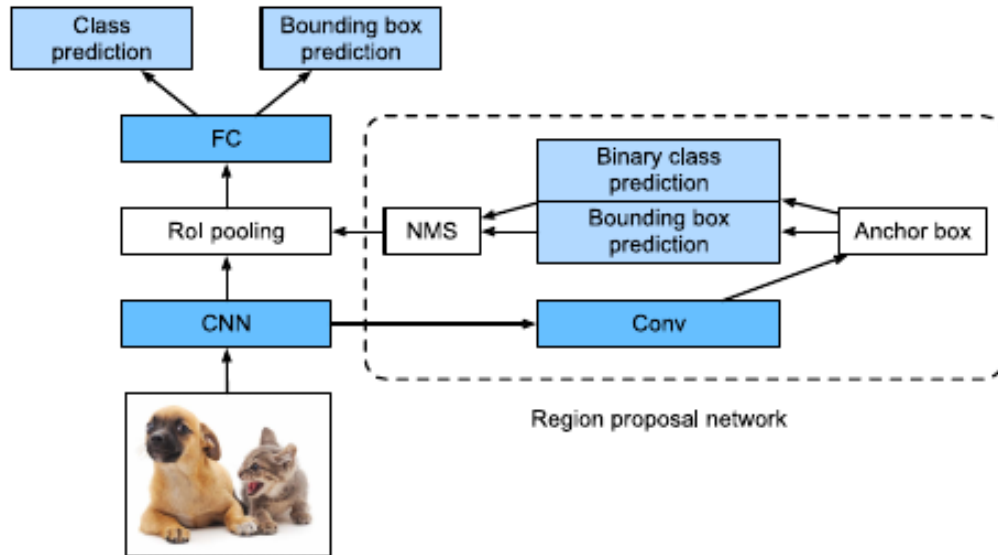


Figure 2.15: Faster R-CNN algorithm.

It is worth noting that, as part of the faster R-CNN model, the region proposal network (steps 2, 3, 4 and 5) is jointly trained with the rest of the model. In other words, the objective function of the faster R-CNN includes not only the class and bounding box prediction in object detection, but also the binary class and bounding box prediction of anchor boxes in the region proposal network. As a result of the end-to-end training, the region proposal network learns how to generate high-quality region proposals, so as to stay accurate in object detection with a reduced number of region proposals that are learned from data.

To end up with this section, Table 2.1 compares the different architectures of the R-CNN for a typical computer vision dataset.

	R-CNN	Fast R-CNN	Faster R-CNN
Test time per image	50 seconds	2 seconds	0.2 seconds
Speed-up	1x	25x	250x
mAP (VOC 2007)	66.0%	66.9%	69.9%

Table 2.1: Comparison between R-CNN, Fast R-CNN, and Faster R-CNN.

2.6 Single Stage Object Detection

Two-stage detectors (such as R-CNN, Fast R-CNN, and Faster R-CNN) work in two phases: first, they generate a set of regions of interest or proposals (region proposals) that likely contain objects, and in a second stage, they refine these proposals and classify them into the appropriate categories. This approach is typically more accurate because it spends more time filtering and fine-tuning the detections, but it is also slower due to the two-stage processing.

In contrast, **single-stage detectors** (such as YOLO) perform detection and classification in a single pass of the network. They divide the image into a grid and directly predict the classes and coordinates of objects in each cell, without an explicit proposal generation step. This makes them much faster and suitable for real-time applications, although they typically had lower accuracy than two-stage models (something recent versions have significantly reduced).

2.6.1 YOLO (v1)

Main Idea

YOLO is the most famous single-stage detector. It is incredibly fast: it can process 45 frames per second. It uses a CNN and then a MLP. The network uses features from the entire image to predict each bounding box. It also predicts all bounding boxes across all classes for an image simultaneously. This means the network reasons globally about the full image and all the objects in the image.

The **algorithm** is the following:

1. The first step starts by dividing the original image into $S \times S$ grid cells of equal shape. If the center of an object falls into a grid cell, that grid cell is responsible for detecting that object.
2. Each grid cell predicts B bounding boxes (typically $B = 2$). In particular, the prediction is

$$Y = (p, x, y, w, h),$$

where p is the probability that the grid contains an object, (x, y) is the center of the bounding box and (w, h) its width and height. Center and dimensions can be relative to the bounding box or to the hole image, but originally the center (x, y) was relative to the bounding box, and dimensions (w, h) to the hole image.

3. In addition to outputting bounding boxes and confidence scores, each cell predicts the class of the object. This class prediction is represented by a one-hot vector of length C , the number of classes in the dataset: $(c_1, \dots, c_C)$.
4. The output of the network is a tensor of shape $S \times S \times (5B + C)$.
5. Select a confidence threshold (for example, only consider region proposals with predicted class with probability greater or equal than 0.5) and use NMS to output the final predictions.
6. At inference time, each box's class specific score is $p \cdot p_{\arg \max\{c_1, \dots, c_C\}}$.

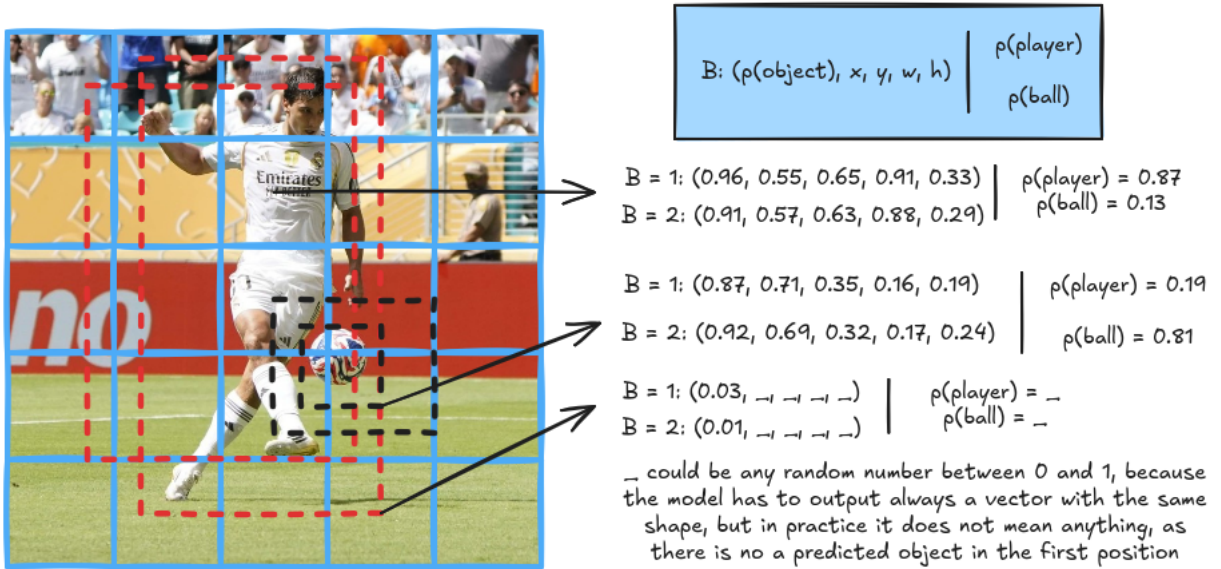


Figure 2.16: Example of a prediction where $B = 2$, $S = 5$ and there are two classes: player (red bounding boxes) and ball (black bounding boxes). The grid is composed by the blue squares. Only the output for three grid cells (marked with arrows) is shown and the coordinates are relative to the whole image, not to each grid cell.

Architecture

The **architecture** of YOLO is shown in Figure 2.17. The initial convolutional layers of the network extract features from the image while the fully connected layers predict the output probabilities and coordinates.

cells that do contain objects. This can lead to model instability, causing training to diverge early on. To remedy this, they increase the loss from bounding box coordinate predictions and decrease the loss from confidence predictions for boxes that do not contain objects. They use two parameters, λ_{coord} and λ_{noobj} to accomplish this. In particular, they used $\lambda_{\text{coord}} = 5$ and $\lambda_{\text{noobj}} = 0.5$.

- Sum-squared error also equally weights errors in large boxes and small boxes. The error metric should reflect that small deviations in large boxes matter less than in small boxes. To partially address this, they predict the square root of the bounding box width and height instead of the width and height directly.

Taking this into account, we will use the following notation:

- We will denote by $\mathbb{1}_{ij}^{\text{obj}}$ that the j -th bounding box of cell i is responsible for predicting an object. A bounding box of a cell is responsible for predicting an object if it is the one with the highest IoU with the ground truth bounding box.
- We will denote by $\mathbb{1}_i^{\text{obj}}$ that the cell i contains an object of any class.
- We will denote by $\mathbb{1}_{ij}^{\text{noobj}}$ that the j -th bounding box is responsible for predicting that there is not an object in that cell. Note that $\mathbb{1}_{ij}^{\text{noobj}} = 1 - \mathbb{1}_{ij}^{\text{obj}}$.

It is important to remark that when predicting there is an object, in each cell we just take into account one of the B predicted bounding boxes, but when predicting there is not an object, we take into account all B predicted bounding boxes. Other interesting comment is that by making predicted bounding boxes of cells responsible for different objects, this leads to specialization between the bounding box predictors. Each predictor gets better at predicting certain sizes, aspect ratios, or classes of object, improving overall recall.

Taking these considerations into account, we use the following notation. The image is divided into S^2 grid cells. Each cell predicts B bounding boxes and one distribution over C classes. The indices

$$i \in \{1, \dots, S^2\}, \quad j \in \{1, \dots, B\}, \quad c \in \{1, \dots, C\}$$

denote a grid cell, a bounding-box predictor, and a class, respectively. Symbols without a hat denote training targets, while symbols with a hat denote model predictions.

For a cell i containing an object, let

$$B_i = (x_i, y_i, w_i, h_i)$$

be its ground-truth bounding box. The j -th bounding-box predictor in that cell produces

$$\hat{B}_{ij} = (\hat{x}_{ij}, \hat{y}_{ij}, \hat{w}_{ij}, \hat{h}_{ij}).$$

The target coordinates require only the cell index i because YOLOv1 assigns one ground-truth object to each cell. Predictions require both i and j because every cell produces B candidate boxes.

The model's confidence prediction for predictor j in cell i is denoted by $\hat{C}_{ij}$. For the responsible predictor, its target is

$$C_{ij} = \text{IoU} \left(\hat{B}_{ij}, B_i \right),$$

although we can use just $C_{ij} = 1$ to simplify the data processing. For a cell that contains no object, the confidence target is $C_{ij} = 0$.

Finally, $p_i(c)$ denotes the target conditional probability for class c in cell i , while $\hat{p}_i(c)$ denotes the corresponding model prediction. When cell i contains an object, the target is:

$$p_i(c) = \begin{cases} 1 & \text{if the object belongs to class } c, \\ 0 & \text{otherwise.} \end{cases}$$

Using this notation, the YOLOv1 loss is

$$\mathcal{L} = \mathcal{L}_{\text{BB}} + \mathcal{L}_{\text{O}} + \mathcal{L}_{\text{NO}} + \mathcal{L}_{\text{C}},$$

where the bounding-box regression loss is

$$\mathcal{L}_{\text{BB}} = \lambda_{\text{coord}} \sum_{i=1}^{S^2} \sum_{j=1}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_{ij})^2 + (y_i - \hat{y}_{ij})^2 + \left(\sqrt{w_i} - \sqrt{\hat{w}_{ij}} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_{ij}} \right)^2 \right],$$

the confidence loss for the responsible predictor is

$$\mathcal{L}_{\text{O}} = \sum_{i=1}^{S^2} \sum_{j=1}^B \mathbb{1}_{ij}^{\text{obj}} \left(C_{ij} - \hat{C}_{ij} \right)^2,$$

the confidence loss for cells without objects is

$$\mathcal{L}_{\text{NO}} = \lambda_{\text{noobj}} \sum_{i=1}^{S^2} \sum_{j=1}^B \mathbb{1}_{ij}^{\text{noobj}} \hat{C}_{ij}^2 = \lambda_{\text{noobj}} \sum_{i=1}^{S^2} \sum_{j=1}^B \left(1 - \mathbb{1}_{ij}^{\text{obj}} \right) \hat{C}_{ij}^2,$$

and the classification loss is

$$\mathcal{L}_{\text{C}} = \sum_{i=1}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c=1}^C (p_i(c) - \hat{p}_i(c))^2.$$

The bounding-box and object-confidence terms are evaluated only for the responsible predictor. The no-object confidence term is evaluated for all non-responsible predictors of the cells, and the classification term is evaluated once per cell containing an object.

2.7 Exercises

Exercise 1: Single Object Detection

Train a model to detect a single object using the procedure described in the first section. You can download a dataset from the internet or create a small custom dataset yourself (e.g., detecting your own shoes in various photos). Ensure your loss function properly balances the classification loss and the bounding box regression loss.

Exercise 2: Naive Proposal Generation

Write a script that generates c random bounding boxes for each pixel in a given image. Create a visualization function to overlay a random subset of these boxes onto the original image. Discuss the computational inefficiency of this naive approach.

Exercise 3: Intersection over Union (IoU)

Implement a function from scratch to calculate the IoU for two given bounding boxes. Your function must include a parameter to seamlessly switch between the two standard bounding box representations: (x_1, y_1, x_2, y_2) and the center-based (x, y, w, h) .

Exercise 4: Non-Maximum Suppression (NMS)

Write a function to perform NMS given a list of predicted bounding boxes, their corresponding confidence scores, and a predefined IoU threshold ε . Test your function on a dummy list of heavily overlapping boxes to ensure only the highest-confidence predictions remain.

Exercise 5: Mean Average Precision (mAP)

Create a function to calculate the mAP given a set of predicted bounding boxes, ground truth bounding boxes, and an IoU threshold. Your function should output the precision-recall curve for each class and compute the final numerical integration (e.g., using the trapezoidal rule with `numpy`).

Exercise 6: Anchor Box Generation

Create a function to generate anchor boxes for an image given a list of scales $s_1, \dots, s_n$ and aspect ratios $r_1, \dots, r_m$. Calculate the exact number of bounding boxes generated for an image of shape $H \times W$ using only the combinations containing the reference scale s_1 or reference ratio r_1 .

Exercise 7: Region of Interest (RoI) Pooling

Implement a function to perform RoI pooling on an input feature map. Your function should take a tensor of shape $C \times H \times W$ and output a tensor of shape $C \times h \times w$, properly handling the grid cell division (using $\lceil H/h \rceil \times \lceil W/w \rceil$) and the aggregation function (e.g., max pooling).

Exercise 8: Architectural Implementations

Write the forward pass pseudocode (or PyTorch code) for both an R-CNN and a Fast R-CNN from scratch. For the R-CNN, highlight the bottleneck of independent feature extraction. For the Fast R-CNN, demonstrate how the CNN is applied only once to the entire image before RoI pooling.

Exercise 9: Architectural Variations

Propose alternative machine learning algorithms for the classification and bounding box regression steps in the R-CNN family. For example, how would you integrate a Random Forest Regressor or a Gradient Boosting Classifier into the Fast R-CNN pipeline? Discuss the potential impacts on training time and backpropagation.

Exercise 10: YOLO Loss Function

Analyze the YOLO loss function $\mathcal{L}$. Explain mathematically why the square root is taken for the width and height in the bounding box regression term $\mathcal{L}_{\text{BB}}$. How does this design choice penalize small errors in small boxes differently than small errors in large boxes?

Exercise 11: Two-Stage vs. Single-Stage Detectors

Write a brief essay discussing the primary advantages and disadvantages of single-stage detectors (like YOLO) versus two-stage detectors (like Faster R-CNN). In what real-world scenarios would you strictly prefer one over the other?

